FULL STACK DEVELOPMENT WITH MERN STACK

PROJECT DOCUMENTATION

STOCK TRADING APP (SB Stocks)

1. Introduction

Project Title: STOCK TRADING APP

Description: STOCK TRADING APP, branded as SB Stocks, is a full-stack web application developed using the MERN stack. It provides a simulated stock-trading environment where users can register and log in securely, view available and trending stocks, inspect stock charts, place simulated buy and sell orders, manage a portfolio, review order history, and manage wallet funds. The application also includes role-based administration features for managing users, stocks, orders, and transactions.

The project is designed as an educational trading platform and uses virtual funds rather than real-money transactions.

2. Project Overview

Purpose: The purpose of the application is to provide a simple and practical environment for learning stock-market workflows without financial risk. It demonstrates end-to-end MERN development by integrating a React-based frontend, REST APIs built with Node.js and Express.js, and MongoDB for persistent data storage.

Key Features:

- Secure user registration and login with role-based access.
- Stock watchlist and trending-stock display.
- Stock search and detailed stock view with chart visualization.
- Simulated stock buying and selling with quantity and price calculation.
- Portfolio management showing owned stocks, quantities, buy prices, and total values.
- Order history for tracking buy and sell transactions.
- User profile and virtual wallet with add-funds, withdraw, and transaction-history functions.
- Admin dashboard for centralized management.
- Admin management of users, stocks, orders, and transaction records.

3. System Architecture

Frontend

The frontend is developed with React.js and provides the user interface for authentication, stock browsing, trading simulation, portfolio management, order history, profile and wallet operations, and administrative functions. React Router is used for navigation, while Axios supports communication with backend APIs.

Technologies: React.js, Vite, JavaScript, HTML5, CSS3, React Router DOM, Axios.

Backend

The backend is built with Node.js and Express.js. It handles REST API requests, authentication and authorization, user operations, stock management, simulated order processing, portfolio-related operations, wallet transactions, and administrative functions. JWT is used for protected access and bcryptjs is used for secure password hashing.

Technologies: Node.js, Express.js, JWT, bcryptjs, CORS, dotenv.

Database

MongoDB is used as the application's database, with Mongoose providing schema modeling and database interaction. The database stores application data such as user accounts, roles, stocks, orders, portfolio/trading information, and wallet transactions.

4. Setup and Installation

Prerequisites:

- Node.js (v16 or above)
- npm
- MongoDB
- Git
- Visual Studio Code
- Postman
- Google Chrome or another modern browser

Installation: Clone the project repository, install the required dependencies separately in the client and server directories, configure the environment variables (including the MongoDB connection string and JWT secret), start MongoDB if a local database is used, run the backend server, and then start the Vite development server for the frontend.

Typical frontend dependencies: axios, react-router-dom.

Typical backend dependencies: express, mongoose, bcryptjs, jsonwebtoken, cors, dotenv.

5. Project Structure

Client (React Frontend): Contains frontend source files, reusable components, pages, routing, styling, assets, and API communication logic.

Server (Node.js Backend): Contains configuration, controllers, models, routes, middleware, environment configuration, and the main server entry point.

The separation of client and server components supports maintainability and allows the frontend, backend, and database layers to be developed and tested independently.

6. User Interface and Functional Modules

Landing Page: Introduces SB Stocks and provides navigation to login and registration.

Login and Registration: Allows users to create accounts and authenticate securely.

Home / Watchlist: Displays available stocks and trending stocks, with options to view charts and initiate simulated purchases.

Stock Trading View: Displays stock information and chart visualization and allows users to select quantity and perform simulated buy or sell operations.

Portfolio: Displays stocks currently held by the user and provides access to stock charts and selling actions.

Order History: Maintains a record of the user's simulated buy and sell orders.

Profile and Wallet: Displays user details and virtual balance and supports adding or withdrawing virtual funds and viewing wallet transactions.

Admin Dashboard: Provides administrative access to user, stock, order, and transaction management.

7. Testing

Tools Used: Postman, Google Chrome, Visual Studio Code, MongoDB, and manual frontend testing.

Testing Strategy: Backend APIs are verified through API request and response testing. Functional testing covers registration, login, stock viewing, simulated buy/sell operations, portfolio updates, order history, wallet operations, and admin functions. Integration testing verifies communication between the React frontend, Express backend, and MongoDB database. User-interface testing verifies navigation, forms, tables, and role-based access.

8. Known Issues and Limitations

1. The application is intended for stock-trading simulation and educational use; it does not execute real financial-market transactions.
2. Some displayed market values may be simulated, manually maintained, or dependent on the configured data source rather than guaranteed live exchange prices.
3. The application requires the backend and database services to be available for full functionality.
4. The current interface is primarily optimized for desktop usage and may require further refinement for smaller mobile screens.

9. Future Enhancements

1. Integration with a reliable real-time market-data API.
2. Advanced interactive charts, technical indicators, and analytics.
3. Improved responsive design and dedicated mobile support.
4. Cloud deployment of the frontend, backend, and database.
5. Enhanced portfolio analytics, profit/loss calculations, and performance reports.
6. Notifications and alerts for price movements and order events.
7. Additional security, automated testing, and production-level monitoring.

10. Conclusion

The STOCK TRADING APP (SB Stocks) demonstrates the development of a complete MERN-stack application that combines secure authentication, stock browsing, simulated trading, portfolio and order tracking, virtual wallet management, and administrative controls. The project provides practical experience in

frontend development, REST API design, database integration, authentication, role-based access, and full-stack application testing.

11. Appendix

GitHub Repository: https://github.com/shannu-122299/StockTradingApp

Project Demo: https://drive.google.com/drive/folders/1u5rbfHrvI8fZiSUP78EoGLL06BJtofHD

Note: Project screenshots and demo media are maintained separately in the repository to keep this documentation concise.